hypercluster

End-to-End Kubernetes Lifecycle Management

Kubespray-powered · Cilium-native · KubeVirt-ready · SSH-first

Susant Sahani

Bash | Kubespray + Ansible | Cilium CNI | KubeVirt

The Problem

Deploying and maintaining a production Kubernetes cluster is still **painful**:

- **Tool sprawl** — kubeadm + Ansible + Helm + custom scripts
- **Inventory hell** — YAML by hand, per-cluster, error-prone
- **Day-2 forgotten** — scaling and upgrades need separate runbooks
- **kubeconfig chaos** — scattered, unpatched, not merged
- **No local testing** — cloud VMs or expensive hardware needed
- **No VM workloads** — KubeVirt setup left as an exercise

Current reality:

10+ commands to get a cluster

✗ No single lifecycle tool

The Solution: hypercluster

> **One config file. One command. Full lifecycle.**

Without hypercluster	With hypercluster
❌ Clone Kubespray manually	✅ Auto-cloned and version-locked
❌ Hand-write hosts.yaml + group_vars	✅ Generated from a single IP list
❌ Run Ansible playbooks by hand	✅ `hypercluster deploy`
❌ kubeconfig scatter + manual patching	✅ Auto-fetched, IP-patched, merged
❌ Scale = edit inventory + re-run	✅ Edit conf, `hypercluster scale`
❌ KubeVirt = separate multi-step process	✅ `KUBEVIRT_ENABLED=true` in conf

At a Glance

1 Config File	**9** CLI Commands	**0** Cloud Provider Needed	**v1.32+** Kubernetes Versions
Cilium Default CNI	**KubeVirt** VM Workloads	**SSH** Only Access Method	**Air-gap** Capable

Kubespray · Ansible · Cilium · Hubble · KubeVirt · CDI · k3s · kubeadm · Podman · containerd

Architecture

```
cluster.conf ──► hypercluster ──► Kubespray
  │
  ├─ clone Kubespray (version-locked)
  ├─ generate hosts.yaml
  ├─ generate group_vars
  ├─ run cluster.yml
  ├─ fetch + patch kubeconfig
  └─ install KubeVirt + CDI
```

Repository Layout

```
hypercluster/
  hypercluster        # CLI entry point
  cluster.conf        # Your config (one file)
  lib/
    log.sh            # Emoji logging
    preflight.sh      # SSH + tool checks
    inventory.sh      # hosts.yaml generation
    kubespray.sh      # Clone + run playbooks
    kubeconfig.sh     # Fetch + patch + merge
  scripts/
    k8s-deploy.sh     # Standalone k3s/kubeadm
    setup-macos.sh    # macOS dev setup
    test-podman.sh    # Local container testing
```

What hypercluster Does NOT Do

> hypercluster is a **thin shell wrapper**.
> It does *zero* Kubernetes work itself.
>
> Every operation delegates to a
> **Kubespray Ansible playbook**.

Nodes Required

- Any bare-metal or VM with Linux
- SSH access (key auth, root)
- No cloud provider dependency
- Works air-gapped

Tested On

AlmaLinux 9 · Rocky 9 · Ubuntu 22.04 · RHEL 9 · Debian 12

One Config File

```
# cluster.conf
CLUSTER_NAME="prod-k8s"
KUBESPRAY_VERSION="v2.30.0"

# Kubernetes
KUBE_VERSION="v1.32.0"
CONTAINER_MANAGER="podman"
CNI_PLUGIN="cilium"

# Nodes (space-separated IPs)
CONTROL_PLANE_IPS="10.0.1.10"
WORKER_IPS="10.0.1.11 10.0.1.12 10.0.1.13"

# SSH
SSH_USER="root"
SSH_KEY="$HOME/.ssh/id_cluster"
SSH_PORT=22

# Add-ons
METRICS_SERVER_ENABLED="true"
INGRESS_NGINX_ENABLED="false"

# KubeVirt — VM workloads
KUBEVIRT_ENABLED="true"
KUBEVIRT_VERSION="v1.3.1"
CDI_VERSION="v1.59.0"

# Output
KUBECONFIG_OUTPUT="$HOME/.kube/prod-k8s.yaml"
```

Then just run:

```
# Check everything first
./hypercluster preflight

# Full cluster bootstrap
./hypercluster deploy

# That's it. You get:
# ✅  Kubernetes cluster
# ✅  Cilium CNI + Hubble
# ✅  kubeconfig auto-fetched
# ✅  KubeVirt operator
# ✅  CDI for disk images
```

No Ansible knowledge required.
No inventory.ini, no group_vars, no playbook flags.
All of that is generated automatically.

Command Reference

Command	What it does
`hypercluster init`	Clone Kubespray (version-locked) and generate `hosts.yaml` + `group_vars`
`hypercluster preflight`	Verify Ansible, Python, kubectl are installed; check SSH key permissions and node reachability
`hypercluster deploy`	Full bootstrap: preflight → init → `cluster.yml` → kubeconfig → KubeVirt
`hypercluster scale`	Add worker nodes listed in conf — runs `scale.yml`, existing workloads untouched
`hypercluster upgrade`	Rolling Kubernetes upgrade via `upgrade_cluster.yml` — one node at a time
`hypercluster remove-node <name>`	Gracefully drain and remove a node; `-f` for ungraceful removal
`hypercluster fetch-kubeconfig`	Pull `admin.conf`, patch 127.0.0.1 → master IP, save + merge into `~/.kube/config`
`hypercluster install-kubevirt`	Deploy KubeVirt operator + CDI on an existing cluster (also runs post-deploy automatically)
`hypercluster status`	Run `kubectl get nodes -o wide` against the cluster
`hypercluster destroy`	Run `reset.yml` — tears down cluster (requires typed confirmation)

Cilium — Default CNI

hypercluster deploys **Cilium** as the default CNI. eBPF-powered networking, security, and observability — out of the box.

Why Cilium

Feature	Cilium	Calico/Flannel
Dataplane	**eBPF**	iptables
Network Policy	L3–L7	L3–L4
Observability	Hubble	None built-in
Service Mesh	Native	No
Performance	High	Medium

Hubble UI — Network Observability

- Real-time flow visualization
- Per-namespace traffic maps
- Policy verdict monitoring
- Service dependency graphs

Cilium CLI — Installed by Default

```
# Check Cilium status
cilium status

# Connectivity test
cilium connectivity test

# Enable Hubble UI
cilium hubble enable --ui

# Port-forward Hubble
kubectl -n kube-system \
  port-forward svc/hubble-ui 12000:80
```

What Kubespray Generates

```
# group_vars/k8s_cluster/k8s-cluster.yml
kube_network_plugin: cilium
cilium_hubble_enabled: true
cilium_enable_bpf_masquerade: true
```

To switch CNI: set `CNI_PLUGIN="calico"` in `cluster.conf`

KubeVirt — VM Workloads on Kubernetes

Run **Windows and Linux VMs** alongside containers
on the same Kubernetes cluster — using familiar `kubectl`.

What Gets Installed

Component	Purpose
KubeVirt Operator	VM lifecycle management
CDI	Containerized Data Importer — disk image uploads
virt-launcher	Pod wrapping each VM (QEMU inside)
virtctl (CLI)	VM console, VNC, SSH, start/stop

Supported VM Types

Ubuntu · RHEL / Rocky · AlmaLinux · CentOS · Windows 10/11 · Windows Server · Any QEMU-compatible image

Enable in cluster.conf

```
KUBEVIRT_ENABLED="true"
KUBEVIRT_VERSION="v1.3.1"
CDI_VERSION="v1.59.0"
```

Or install standalone

```
./hypercluster install-kubevirt
```

Manage VMs with virtctl

```
# Start / stop
virtctl start my-windows-vm
virtctl stop my-ubuntu-vm

# Connect to serial console
virtctl console my-ubuntu-vm

# SSH via VirtIO
virtctl ssh fedora@my-vm

# VNC (graphical)
virtctl vnc my-windows-vm
```

Standalone: k8s-deploy.sh

A **single-file bash deployer** — no Kubespray, no Ansible, no Python. Ideal for quick k3s clusters or when Kubespray is overkill.

Configure

```
# Edit top of script
CLUSTER_NAME="prod"
MASTER_IP="10.0.1.10"
MASTER_USER="root"
MASTER_SSH_KEY="~/.ssh/id_rsa"
WORKERS=(
  "10.0.1.11|root|~/.ssh/id_rsa"
  "10.0.1.12|root|~/.ssh/id_rsa"
)
K8S_DISTRO="k3s"
SSH_PORT=22
```

Run

```
# Basic run
./scripts/k8s-deploy.sh

# External config (no edit)
HC_CONFIG=my.conf \
  ./scripts/k8s-deploy.sh

# Verify SSH only
./scripts/k8s-deploy.sh --verify

# Help
./scripts/k8s-deploy.sh --help
```

Features

- **k3s** or **kubeadm**
- Parallel worker install
- OS auto-detection (cache)
- Podman / Docker / none
- kubeconfig fetch + merge
- Idempotent — re-run safe
- Bash 5 re-exec on macOS

Tested: AlmaLinux 9.7 with k3s v1.34
Node Ready in under 90s.

Day-2 Operations

Scale — Add Nodes

```
# cluster.conf
WORKER_IPS="10.0.1.11 10.0.1.12
            10.0.1.13 10.0.1.14"

# Run scale
./hypercluster scale
```

Kubespray's `scale.yml` only touches **new nodes** — existing workloads are not disrupted.

Upgrade — Rolling

```
# cluster.conf
KUBE_VERSION="v1.33.0"

# Run upgrade
./hypercluster upgrade
```

Kubespray drains each node before upgrading — pods are rescheduled, not killed.

Remove Node

```
# Graceful drain + remove
./hypercluster remove-node worker3

# Force (no drain)
./hypercluster -f remove-node worker3
```

kubeconfig Management

```
# Fetch from cluster
./hypercluster fetch-kubeconfig

# Merge all clusters
./hypercluster merge-kubeconfig

# Use it
export KUBECONFIG=\
  ~/.kube/prod-k8s.yaml
```

macOS Developer Setup

One-Command Setup

```
./scripts/setup-macos.sh
```

What Gets Installed

Tool	Purpose
bash 5	macOS ships 3.2 — needs 4.3+
ansible	via Homebrew (not pip)
kubectl	Cluster access
helm	Chart management
cilium CLI	CNI management
virtctl	KubeVirt VM management
podman	Local testing + container runtime

Local Testing — Podman Containers

```
# Full end-to-end k3s test
# (no cloud, no VMs, no sudo)
./scripts/test-podman.sh

# SSH connectivity check only
./scripts/test-podman.sh --preflight

# Tear everything down
./scripts/test-podman.sh --clean
```

3 containers: 1 master + 2 workers
Each runs **Ubuntu 22.04 + systemd** as PID 1
Identical service management to real nodes
Port-forwarded SSH + k3s API (6443)

Comparison

	hypercluster	Raw Kubespray	kubeadm	k3s	Managed K8s
Config complexity	✅ Single file	⚠ Multi-YAML	❌ Multi-step	✅ Env vars	✅ UI/API
Day-2 ops	✅ Built-in	⚠ Manual playbooks	❌ Manual	❌ Manual	✅ Managed
kubeconfig mgmt	✅ Auto	❌ Manual	❌ Manual	❌ Manual	✅ Auto
Air-gap support	✅ Yes	✅ Yes	⚠ Partial	✂ Limited	❌ No
KubeVirt built-in	✅ Yes	❌ No	❌ No	❌ No	❌ No
Cilium by default	✅ Yes	⚠ Manual	⚠ Manual	⚠ Manual	⚠ Varies
Production grade	✅ Yes	✅ Yes	✅ Yes	⚠ Lightweight	✅ Yes

hypercluster

"From bare nodes to running cluster — and every operation after."

One config file	All cluster config in one place
Full lifecycle	Deploy, scale, upgrade, remove, destroy
Cilium default	eBPF networking + Hubble observability
KubeVirt built-in	Windows + Linux VMs alongside containers
SSH-first	No cloud provider, no agents, works air-gapped
Verified	AlmaLinux 9.7, k3s v1.34, Cilium, KubeVirt live
Kubespray-backed	Battle-tested Ansible, upstream Kubernetes

GitHub: github.com/ssahani/hypercluster | **Author:** Susant Sahani | **Stack:** Kubespray · Cilium · KubeVirt · Podman